```
const mongoose = require('mongoose');

const connectToDB = () => {
  console.log('MONGO_DB value:', process.env.MONGO_DB); // ← add this line
  mongoose
    .connect(process.env.MONGO_DB)
    .then(() => console.log('Connected to MongoDB'))
    .catch((err) => {
      console.error('MongoDB connection error:', err);
      process.exit(1);
    });
};

module.exports = connectToDB;
```

Advantages of Using a Config Folder

- **Centralized configuration:** All settings are kept in one place.
- **Security:** Sensitive information stays in .env instead of source code.
- **Maintainability:** Configuration changes don't affect application logic.
- **Reusability:** Configuration files can be imported wherever needed.
- **Scalability:** Easy to add new configuration files (email, cloud storage, authentication, etc.) as the project grows.

The **config** folder is an important part of a MERN application's backend. It stores configuration files such as the database connection and environment settings, keeping the code modular, secure, and easy to maintain. By placing configuration logic in dedicated files (like config/db.js) and storing sensitive values in a .env file, developers can build applications that are more organized, scalable, and production-ready.

What is a Config Folder?

The **config** folder contains files that configure your application's settings, such as:

- Database connection
- Environment variables
- API keys
- Server configuration
- Authentication settings

Using a config folder makes the project easier to maintain and update.

Without a config folder, all configuration code would be written inside server.js, making the file lengthy and difficult to manage.

Using a config folder provides:

- Better project organization
- Easy maintenance
- Code reusability
- Improved security
- Cleaner code structure